

## Sugar, Sugar — Technical Documentation

### 1. Choice of language and graphics library

**Python** was chosen for its simplicity of prototyping, its object-oriented support (sufficient for this project's needs), and its portability (the same code runs unmodified on Windows, Linux, or macOS). No feature of the game required the raw performance of a compiled language — the computational load (a few dozen to a few hundred grains tested at 30 FPS) comfortably fits within the capabilities of an interpreted language.

#### Threading and multiprocessing considered, then rejected

Parallelizing the grain-testing step (one thread or process per batch of grains, for instance) was considered as a way to anticipate a heavier load, then dropped for two distinct reasons depending on the approach:

- **Threading:** in Python, the *Global Interpreter Lock* (GIL) guarantees that only one thread executes Python bytecode at any given time, even on a multi-core machine. For work that is mostly CPU-bound (testing positions, comparing colors, moving references around the grid), splitting the load across several threads therefore brings **no real parallelism** — the GIL serializes execution regardless. Worse, the synchronization overhead between threads (locks, context switches) would likely have slowed the physics loop down rather than sped it up. Threading in Python is only genuinely useful for work blocked on I/O (network, disk), which doesn't match any of the game's needs.
- **Multiprocessing:** this approach does bypass the GIL (each process runs its own interpreter), but at the cost of fully separate memory spaces. The level grid (up to 1600×900 cells, holding live object references) would need to be serialized and transferred between processes on *every frame* to stay consistent — a communication cost that would far outweigh the computation time it would save, especially at this project's scale (a few hundred grains at most). Multiprocessing is worthwhile for large batches of independent work processed once, not for a continuously-updated shared-state simulation like this one.

A single-threaded, single-process simulation loop therefore remains both the simplest and the most performant option given these constraints.

**Pygame** was chosen as the graphics library for several reasons:

- A simple, low-level API (surfaces, blit, rectangle-based collision detection) sufficient for a 2D grid-based game, without the overhead of a full engine (Unity, Godot) that this project's scale doesn't warrant.
- Native handling of sound (pygame.mixer), keyboard/mouse events, and rendering, with no extra dependency required.

- Strong documentation and community, reducing the risk of getting stuck on an obscure issue.
- Lightweight to distribute: combined with PyInstaller, it produces a standalone executable without requiring Python to be installed on the end user's machine.

## 2. General architecture

main.py → instantiates Audio, Display, GameManager

src/

game\_manager.py → navigation state machine (home / settings / selection / game)

game\_loop.py → simulation loop for a single level (physics, drawing, input)

screen\_manager.py → a single pygame event poll per frame, shared between GameManager and GameLoop

display.py → rendering, asset loading, scaling/resolution

audio.py → music playback

level.py → level loading (JSON), Level class (state of a level in progress)

sugar.py → Sugar class (a single grain)

cup.py → Cup class (cup, with per-color fill requirements)

wall.py → Wall class (JSON obstacles and mouse-drawn walls)

portal.py → Portal class (color changer)

save.py → player progress persistence (cache/save.json)

settings.py → global constants (resolution, colors, FPS...)

cache/

level\_data.json → data for every level

save.json → player progress (auto-created if missing)

assets/ → images and music

The navigation flow is a simple state machine: GameManager loops over a current page ("homepage", "setting", "level", "game") and fully delegates the "game" page to a GameLoop instance, which runs its own simulation loop for the duration of the session.

## 3. Data structures and objects

### The grid

Each level holds a 2D grid (a list of list), matching the size of the logical rendering surface (1600×900): **one grid cell = one logical pixel**. Each cell holds either None (empty) or a direct reference to the object occupying it (Sugar, Wall, Cup, Portal). This choice — object references rather than numeric codes — avoids needing a separate lookup table and keeps every cell immediately usable (isinstance(cell, Cup), etc.).

### Grains (Sugar)

Every grain, whether still moving or at rest, is kept in a **single list** (Level.\_\_grains) and is **always present in the grid** at its current position — including while it is falling. This wasn't the initial design: an earlier version distinguished "active" grains (never registered in the grid) from "settled" ones (registered permanently once). That model caused a bug after inverting gravity — a grain put back into motion could remain incorrectly stuck, invisible to grains tested in the same frame, since it wasn't registered in the grid while moving. The unified model fixes this: every move updates the grid immediately (old cell freed, new cell occupied), so two grains processed in the same frame correctly see each other, regardless of processing order.

### Walls (Wall)

A Wall is defined by a shape (rect, circle, triangle, or line for mouse-drawn strokes) and its geometric parameters. Rasterization (converting the vector shape into grid cells) happens once, at load time or at the moment of drawing — never recomputed every frame. A separate list (Level.\_\_walls) keeps only the original shapes, for efficient rendering (a handful of pygame.draw calls per frame rather than scanning the whole grid).

### Cups (Cup)

A cup occupies a square of cells: only the top row (mouth\_cells()) holds the Cup reference itself (the absorption zone); the other three sides are built as ordinary solid Wall objects. A cup carries a requirements dictionary (color → amount required) and a filled one (color → amount received so far), which natively supports both single-color and dual-color cups with the same structure.

### Portals (Portal)

Like cups, a portal occupies a square; its left and right sides are solid (Wall), while the top and bottom stay passable from both directions. A Level.\_\_portal\_cells dictionary (position → Portal) **restores** the portal's reference in the grid as soon as a grain that was passing through it moves away — without this, the first grain to pass through would progressively "erase" the portal cell by cell.

### Level format (JSON)

All levels are described in a single file, `cache/level_data.json`, shaped as `{"level-01": {...}, "level-02": {...}, ...}`. Each level specifies its spawn point, default sugar color, grain quota, obstacles, cups (simple format `{"color": ..., "capacity": ...}` or multi-color `{"colors": {...}}`), and any portals.

#### 4. Gravity and collision management

Every frame, for each grain:

1. **Vertical direction:** `target_y = y + gravity` (gravity is 1 or -1, changeable in-game). If this position falls outside the grid, the opposite edge is tested instead — teleportation only happens if that cell is actually free; otherwise the grain behaves as if blocked by a wall (this natively implements floor/ceiling-hole levels, with no dedicated code for that mechanic).
2. **Three horizontal directions tested** at that `target_y`: straight down, diagonal left, diagonal right (in that order, unbiased). For each:
  - out of horizontal bounds → grain destroyed (side-border loss, as specified in the brief);
  - empty cell → valid candidate;
  - Cup → attempt to absorb (`catch_grain`); if accepted, the grain is destroyed and the fill count incremented; if refused, the direction is treated as blocked;
  - Portal → the grain's color is changed, the cell is added to the candidates (passable);
  - anything else (Wall, another grain already there) → blocked.
3. A cell is picked at random (`random.choice`) among the valid candidates; the grain moves there (`Level.move_grain`, which frees the old cell and occupies the new one **before** moving on to the next grain, guaranteeing the intra-frame consistency described above). If no cell is valid, the grain stays put — it is retested, without exception, on the next frame.

This mechanism is deliberately unoptimized (no distinction between "moving" and "at rest" grains survives from one frame to the next): at this project's scale (a few hundred grains at most), the cost is negligible, and this simplicity avoids the inconsistencies a selective wake-up system would introduce.

Collision itself relies entirely on explicit `isinstance()` checks rather than a shared polymorphic method (an `on_contact()` approach was tried, then abandoned) — a deliberate choice to keep every case readable at a glance inside `GameLoop`, at the cost

of one instance check per object type rather than moving that responsibility into each class.

## 5. Display and scaling

Rendering happens on a fixed logical surface (1600×900), independent of the window's actual size. `Display.render()` computes a single scale factor (`pygame.transform.scale`, never `smoothscale`, to keep single-pixel grains sharp rather than blurry) and centers the result with black bars if the ratio doesn't match exactly. Every mouse position is converted back to logical coordinates (`screen_to_logical`) before being compared against button collision rectangles or used to draw a wall — without this conversion, clicks would no longer land in the right place as soon as the window changes size or resolution.

## 6. Save system

`cache/save.json` holds `{"player": {"level": N}}`, where `N` is the number of unlocked levels. On winning a level (`Level.is_complete()`, every cup full), `set_save(level_id)` compares the completed level's position (in alphabetical order of its identifier) to `N`: if the completed level is exactly the last one unlocked, `N` is incremented and the file rewritten. Replaying an already-passed level changes nothing, ruling out any risk of double-incrementing. If `save.json` is missing (first launch, or a file deliberately excluded from the Git repository), it is automatically recreated with a starting value.

## 7. Distribution

The game is packaged with `PyInstaller` (`--onedir` mode):

```
python -m PyInstaller --onedir --windowed --name SupSugar --add-data "assets;assets"
--add-data "cache/level_data.json;cache" main.py
```

`--onedir` (rather than `--onefile`) was chosen because it places the executable and its data in a permanent folder, never deleted between runs — a requirement for `cache/save.json`, generated next to the executable, to survive the game being closed.

## 8. Known limitations and possible improvements

- Resolution and music settings are not persisted between launches (unlike level progress).
- `GameManager` and `GameLoop` share a common `ScreenManager` for click detection, but each still keeps a portion of duplicated navigation logic (`__click_coord`) — a richer `ScreenManager` or a genuinely shared navigation state could reduce this further.

- The build only targets Windows; a Linux/macOS distribution would require re-running PyInstaller from a machine running the target OS (no cross-compilation is possible).